



Aimably Aurora Write Instance Vertical Autoscaling

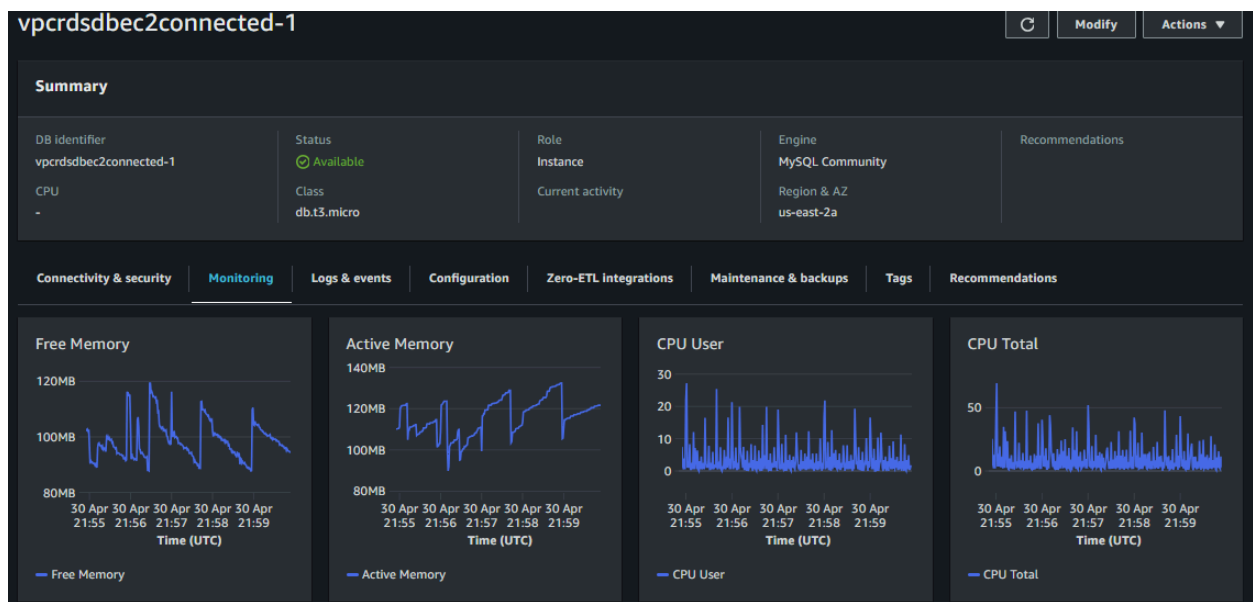
Aimably Aurora Write Instance Vertical Autoscaling is a script-based tool that can automatically increase or decrease the capacity and performance capabilities of Aurora write instances in response to external factors, such as user demand.

Aimably Aurora Write Instance Vertical Autoscaling includes an AWS Lambda Function that uses AWS Aurora native failover mechanisms to facilitate vertical autoscaling of Aurora write instances. The function is triggered based on EventBridge events, which can be triggered by CloudWatch alarms configured to track your chosen performance metrics.

For example, you configure a CloudWatch alarm to alert when the CPU of the Aurora write instance is below 10% for a specific time period, this can be configured to trigger the Aimably Aurora Write Instance Vertical Autoscaling function to scale down to a lower and less expensive instance class. Conversely, you can configure a CloudWatch alarm to alert if the database CPU is at 90% for a specific period of time, which can trigger the Aimably Aurora Write Instance Vertical Autoscaling function to scale up to a higher and more expensive instance class. This enables you to have the correct instance class based on performance, thereby avoiding expensive overprovisioned databases.

Best Practices

- **Utilization Pattern Review:** Because Aurora was not built for vertical autoscaling by default, it is important to become familiar with the utilization patterns of each of your database clusters to select the best parameters for the Aimably Aurora Write Instance Vertical Autoscaling function.
 - This data is readily available in the Monitoring tab of a database in the RDS console. You can use the standard CloudWatch monitoring or the newer Enhanced Monitoring that is available. You may adjust the reporting time between 1 hour and 15 months to use this data to understand the periods where peak performance is required.



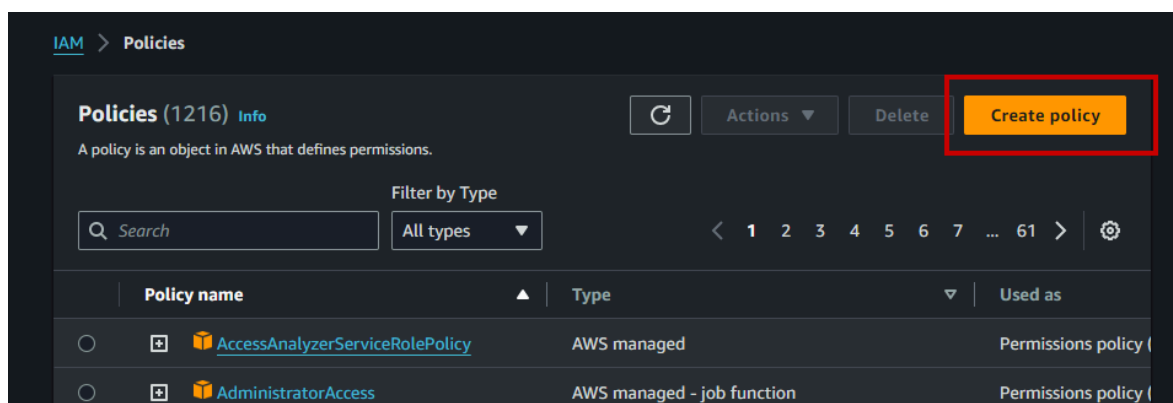
- **Metric Selection:** Every database cluster experiences performance constraints unique to its workload. Select the metrics that become most constrained in order to achieve the best results.
 - Commonly used metrics for an Aurora database are CPU Utilization, Database Connections, Freeable Memory, or Read/Write Latency. You must specify the Write instance for tracking these metrics, rather than the cluster as a whole. You can read more in-depth on the various metrics that can be used for an alarm in this support article, [AWS CloudWatch/RDS Recommended Alarms](#).
- **Threshold Selection:** Aurora failover process takes between 30 and 60 seconds to complete, allowing for finely-tuned metric responsiveness. For the most cost optimized implementation, select alert thresholds that tightly track current needs for capacity rather than predictive needs.
 - In the event of highly predictable traffic patterns, you may also choose to configure events in EventBridge to follow a set schedule instead of using CloudWatch alarms. You can read more in-depth on EventBridge scheduling in this support article, [Creating an EventBridge Rule That Runs on a Schedule](#).
- **Sizing Selection:** The Aimably Aurora Write Instance Vertical Autoscaling function requires that you select two specific instance classes, one to service the lower demand and one to service the higher demand. Make sure to review historical capacity demands and select the instance classes that best represent your needs in both scenarios, ensuring the higher class is not triggered too frequently due to demand on the lower class and that the lower class is not excessively overprovisioned so as to keep costs high.
 - Users may choose to adjust the function to support more than two instance classes.

Prerequisites

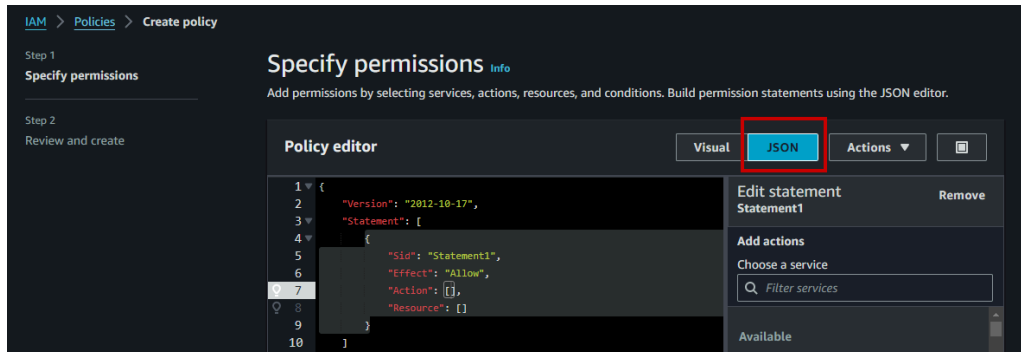
- You need administrator access to your AWS organization

Create IAM Autoscaling Policy

- Navigate to the [IAM Console](#) in AWS
- Click on the Policies menu item in the left-hand side navigation
- Click the Create Policy button in the upper right-hand corner



- Click on the JSON button in the Policy Editor title bar



- Paste the below JSON text into the Policy Editor - be sure to copy it from the first bracket to the last

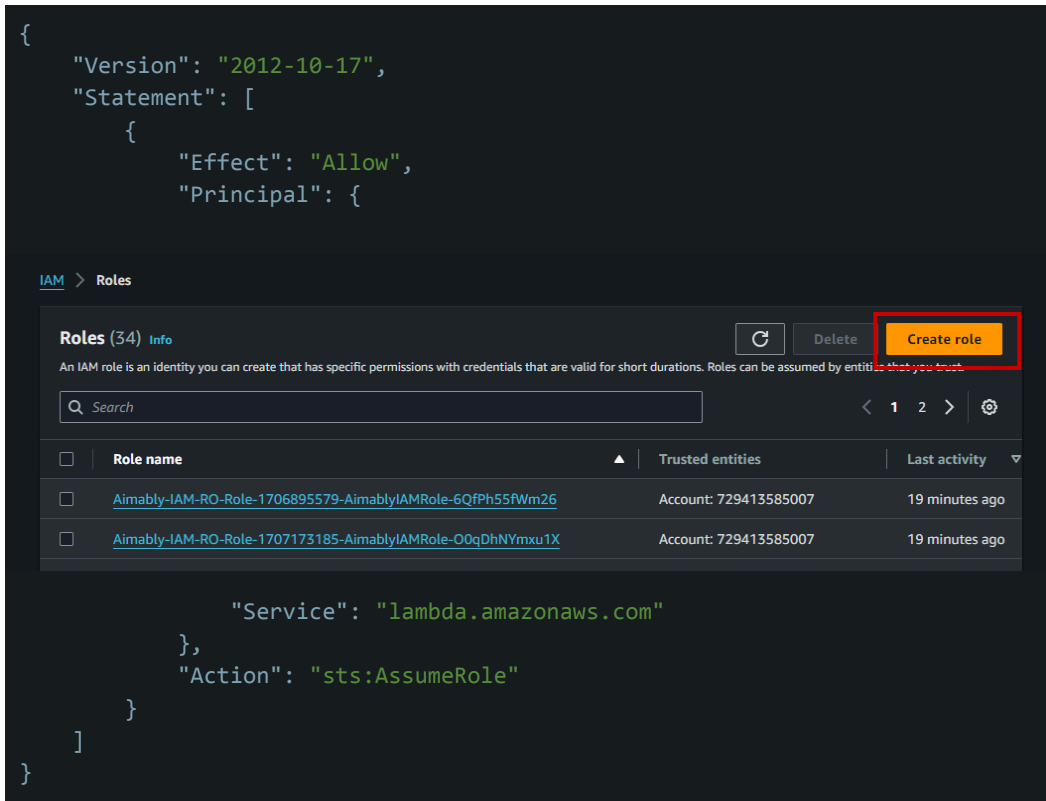
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "rds:AddTagsToResource",
        "rds:RemoveTagsFromResource",
        "ec2:CreateNetworkInterface",
        "ec2:DescribeNetworkInterfaces",
        "ec2>DeleteNetworkInterface",
        "iam:PassRole",
        "rds:CreateDBInstance",
        "rds:CreateDBInstanceReadReplica",
        "rds:CreateBlueGreenDeployment",
        "rds:DescribeBlueGreenDeployments",
        "rds:SwitchoverBlueGreenDeployment",
        "rds>DeleteBlueGreenDeployment",
        "rds>DeleteDBInstance",
        "rds:DescribeDBInstances",
        "rds:ModifyDBInstance",
        "rds:PromoteReadReplica",
        "rds:DescribeDBClusters",
        "rds:FailoverDBCluster",
        "logs:*",
        "lambda:InvokeFunction"
      ],
      "Resource": "*"
    }
  ]
}
```

- Click the Next button
- Name the policy AimablyAuroraVerticalAutoscaling
- Click the Create Policy button

Create IAM Autoscaling Role

- Navigate to the [IAM Console](#) in AWS
- Click on the Roles menu item in the left-hand side navigation
- Click on the Create Role button
- Select the AWS Service radio button
- Under Use Case, select Lambda
- Click Next
- Use the search bar and locate and check the following policies:
 - AimablyAuroraVerticalAutoscaling
 - AWSLambdaBasicExecutionRole
 - AWSLambdaDynamoDBExecutionRole
- Click Next
- Name the role `AimablyAuroraVerticalScalingRole`
- Verify that the text in the Trust Policy matches the below JSON text

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "lambda.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```



Roles (34) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	Aimably-IAM-RO-Role-1706895579-AimablyIAMRole-6QfPh55fWm26	Account: 729413585007	19 minutes ago
☐	Aimably-IAM-RO-Role-1707173185-AimablyIAMRole-00qDhNymxu1X	Account: 729413585007	19 minutes ago

```

    "Service": "lambda.amazonaws.com"
  },
  "Action": "sts:AssumeRole"
}
]
}
```

- Click the Create Role button

Retrieve Source from GitHub

- Use this link to access the Aimably RDS Instance Vertical Autoscaling repository: <https://github.com/mmilbourne/aimablyverticalautoscaling>
- Click on the latest release in the right-hand navigation

aimablyverticalautoscaling Public

Watch 1

Fork 0

Star 0

main 1 Branch 1 Tags

Go to file

Add file

Code

About

	mmilbourne Merge pull request #2 from mmilbourne/FileOrganizationChanges	622f3da · 19 hours ago	6 Commits
	.vscode	Moving the source into a src folder and setting up build scri...	19 hours ago
	src	Moving the source into a src folder and setting up build scri...	19 hours ago
	.gitignore	Initial commit	2 days ago
	LICENSE	Initial commit	2 days ago
	README.md	Update README.md	2 days ago

README GPL-3.0 license

Aimably Vertical Autoscaling

Lambda function to aid in the vertical scaling of RDS database instances and clusters.

Lambda function to aid in the vertical scaling of RDS database instances and clusters.

aimably.com

python aws lambda rds
verticalscaling

Readme

GPL-3.0 license

Activity

0 stars

1 watching

0 forks

Report repository

Releases 1

Initial 1.0 Release Latest
20 hours ago

- Click on the file named `deploy.zip` to download the latest version in a .zip file

Initial 1.0 Release Latest

Compare

mmilbourne released this 20 hours ago · 2 commits to main since this release 1.0 57abe39

First version of the Aimably Vertical Auto Scaling. This supports RDS Instance databases only.

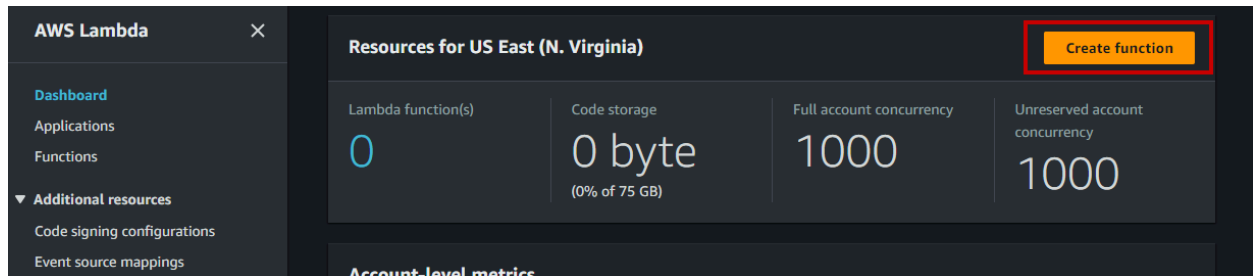
Assets 3

	deploy.zip	38.1 KB	20 hours ago
	Source code (zip)		yesterday
	Source code (tar.gz)		yesterday



Create Lambda Function

- Navigate to the [Lambda Console](#) in AWS
- Click the Create Function button



- Select the Author From Scratch radio button
- Name the function AimablyAuroraVerticalAutoscaling
- Select the latest version of Python for Runtime
- Select x86_64 for Architecture
- Expand the Change Default Execution Role accordion
- Select *Use an Existing Role* radio button
- Select the AimablyAuroraVerticalScalingRole from the Existing Role drop-down
- Expand the Advanced Settings accordion
- Check the checkbox for Enable VPC
- Select the VPC the RDS instance uses in the VPC drop-down
- Select the subnet(s) the RDS instance uses in the Subnets drop-down
- Select the Security Group(s) the RDS instance uses in the Security Groups drop-down

Lambda > Functions > Create function

Create function Info

Choose one of the following options to create your function.

☒ **Author from scratch**
Start with a simple Hello World example.

☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name Info
 Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime Info
 Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Architecture Info
 Choose the instruction set architecture you want for your function code.
☒ x86_64
☐ arm64

Permissions Info
 By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

[▶ Change default execution role](#)

▼ Advanced settings

☐ **Enable Code signing** Info
 Use code signing configurations to ensure that the code has been signed by an approved source and has not been altered since signing.

☐ **Enable function URL** Info
 Use function URLs to assign HTTP(S) endpoints to your Lambda function.

☐ **Enable tags** Info
 A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources, track your AWS costs, and enforce attribute-based access control.

☒ **Enable VPC** Info
 Connect your function to a VPC to access private resources during invocation.

VPC
Choose a VPC for your function to access.

vpc-0913a5c68c52ec02d (172.30.0.0/16)

☐ Allow IPv6 traffic for dual-stack subnets
You can allow outbound IPv6 traffic to subnets that have both IPv4 and IPv6 CIDR blocks.

Subnets
Select the VPC subnets for Lambda to use to set up your VPC configuration.

Choose subnets

subnet-04aa1835bdf603793 (172.30.0.0/24) us-east-1a X

subnet-0575dcd5c29beaba3 (172.30.1.0/24) us-east-1b X

Security groups
Choose the VPC security groups for Lambda to use to set up your VPC configuration. The table below shows the inbound and outbound rules for the security groups that you choose.

Choose security groups

sg-006d18c0d2d4b0022 (default) X
default VPC security group

Inbound rules **Outbound rules**

Security group ID	Protocol	Ports	Source
sg-006d18c0d2d4b0022	All	All	sg-006d18c0d2d4b0022

Cancel **Create function**

- Click the Create Function button
- Once the function has been created, in the Configuration tab choose General Configuration, then the Edit button.

Code Test Monitor **Configuration** Aliases Versions

General configuration info

Triggers

Permissions

Destinations

Function URL

Environment variables

Description

Memory 128 MB

Ephemeral storage 512 MB

Timeout 0 min 3 sec

SnapStart info

None

Edit

- Update the timeout to 5 minutes to allow the states to change during a polling wait period built into the function. Once complete, click Save.

Edit basic settings

Basic settings info

Description - optional

Memory info
Your function is allocated CPU proportional to the memory configured.

128 MB
Set memory to between 128 MB and 10240 MB

Ephemeral storage info
You can configure up to 10 GB of ephemeral storage (tmpfs) for your function. [View pricing](#)

512 MB
Set ephemeral storage (tmpfs) to between 512 MB and 10240 MB

SnapStart info
Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#) for Python and .NET runtimes. [View pricing](#)

None

Supported runtimes: .NET 8 (C#), Java 11, Java 17, Java 21, Python 3.12, Python 3.13

Timeout
5 min 0 sec

Execution role
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

☒ Use an existing role
☐ Create a new role from AWS policy templates

Existing role
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

AimablyEDSVerticalScalingRole

[View the AimablyEDSVerticalScalingRole role](#) on the IAM console.

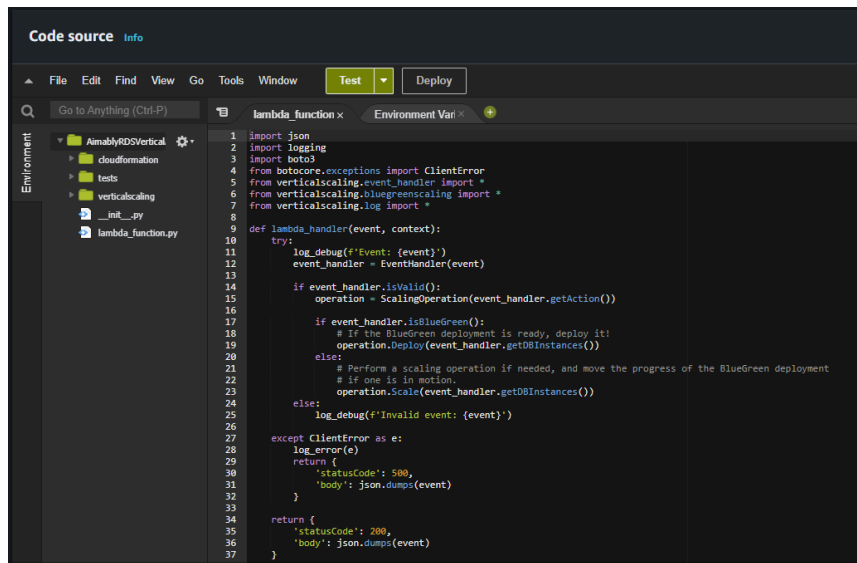
Cancel **Save**

Create Lambda Layer

- Scroll to the bottom of the Lambda Function page
- Click on the Add a Layer button
- Select the AWS Layers radio button
- Select the layer named AWSLambdaPowertoolsPythonV2
- Select the latest version in the Version drop-down
- Click the Add button

Upload the Source Code

- In the Code Source editor, click on the Upload From button and choose .zip file
- Locate the deploy.zip file you downloaded from GitHub previously
- Drag the deploy.zip file into the Upload a .zip File dialog, or click the Upload button and navigate to the file
- Click the Save button
- Verify the code in the Code Source editor matches the below

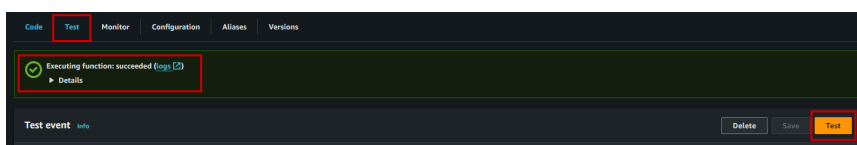


```

1 import json
2 import logging
3 import boto3
4 from botocore.exceptions import ClientError
5 from verticalscaling.event_handler import *
6 from verticalscaling.bluegreenscaling import *
7 from verticalscaling.log import *
8
9 def lambda_handler(event, context):
10     try:
11         log_debug(f'Event: {event}')
12         event_handler = EventHandler(event)
13
14         if event_handler.isValid():
15             operation = ScalingOperation(event_handler.getAction())
16
17             if event_handler.isBlueGreen():
18                 # If the BlueGreen deployment is ready, deploy it!
19                 operation.Deploy(event_handler.getDBInstances())
20             else:
21                 # Perform a scaling operation if needed, and move the progress of the BlueGreen deployment
22                 # if one is in motion.
23                 operation.Scale(event_handler.getDBInstances())
24             else:
25                 log_debug(f'Invalid event: {event}')
26
27     except ClientError as e:
28         log_error(e)
29         return {
30             'statusCode': 500,
31             'body': json.dumps(event)
32         }
33
34     return {
35         'statusCode': 200,
36         'body': json.dumps(event)
37     }

```

- Click on the Test tab above the Code Source editor
- Click the Test button

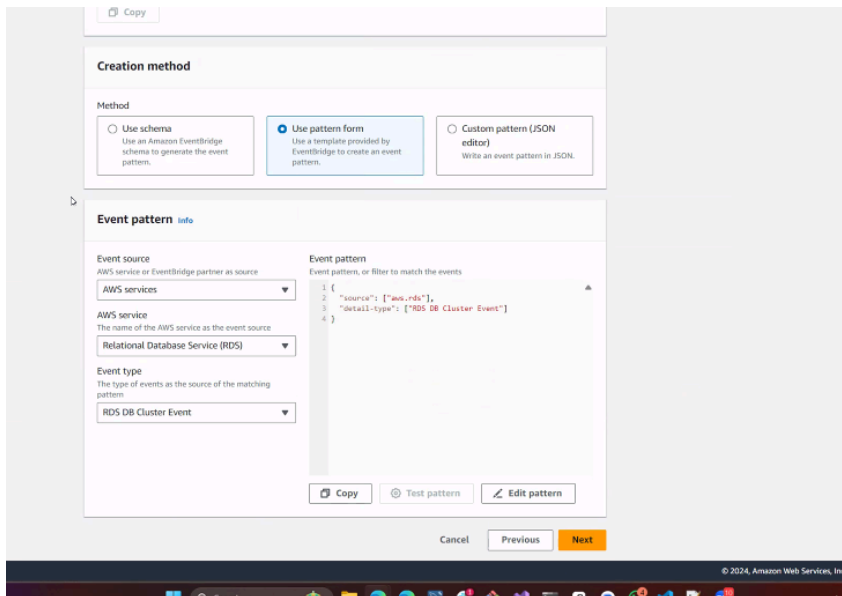


- Verify the Executing function states succeeded, and in the Details "Received an unknown event"

Create EventBridge Rules

You will be creating one EventBridge rule to work with Aimably Aurora Write Instance Vertical Autoscaling. The rules will enable the Lambda function to detect the cluster state changes to complete the autoscaling events. Please note that EventBridge Rules are region specific. As such you will need to repeat the below steps and create both rules in any region you intend to autoscale in.

- RDSClusterStateChange Rule
 - Navigate to the [EventBridge Console](#) in AWS
 - Click on the Rules item in the left-hand navigation
 - Click the Create Rule button
 - Name the rule RDSClusterStateChange
 - Select the default option in the Event Bus drop-down
 - Select to enable the rule on the selected event bus
 - Select the Rule With an Event Pattern radio button
 - Click Next
 - Select Use Pattern Form as the Creation Method
 - Select AWS Services in the Event Source drop-down
 - Select Relational Database Service (RDS) in the AWS Service drop-down
 - Select RDS DB Cluster Event in the Event Type drop-down
 - Click Next



The screenshot shows the 'Create Rule' wizard in the AWS EventBridge console, specifically the 'Event pattern' step. The 'Creation method' section at the top has three options: 'Use schema', 'Use pattern form' (which is selected), and 'Custom pattern (JSON editor)'. Below this, the 'Event pattern info' section contains three dropdown menus: 'Event source' set to 'AWS services', 'AWS service' set to 'Relational Database Service (RDS)', and 'Event type' set to 'RDS DB Cluster Event'. To the right of these dropdowns is a text area showing the resulting event pattern in JSON:

```
{
  "source": ["aws.rds"],
  "detail-type": ["RDS DB Cluster Event"]
}
```

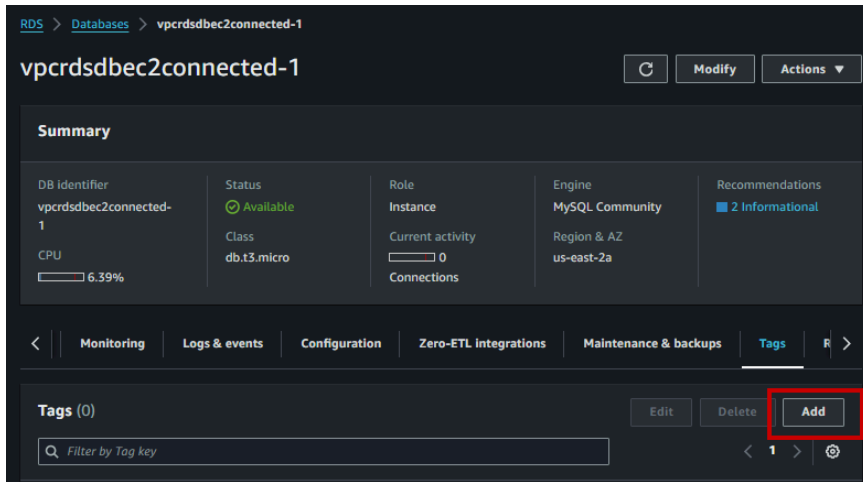
. At the bottom of the form are buttons for 'Copy', 'Test pattern', and 'Edit pattern'. At the very bottom of the wizard are 'Cancel', 'Previous', and 'Next' buttons. The 'Next' button is highlighted in orange.

- Select the AWS service radio button for the Target Type
- Select Lambda Function from the Select a Target drop-down
- Select the AimablyAuroraVerticalAutoscaling Lambda function from the Function drop-down
- Leave all other inputs as they are by default configured
- Click Next
- Add any tags
- Click Next
- Click the Create Rule button

Configure Database(s)

You need to create two tags on any database you want to use Aimably Aurora Write Instance Vertical Autoscaling with. These tags are used by the Lambda function to understand scale up and scale down events.

- Navigate to the [RDS Console](#) in AWS
- Click on the Databases item in the left-hand navigation
- Click on the database to access the configuration
- Click on the Tags tab
- Click the Add button



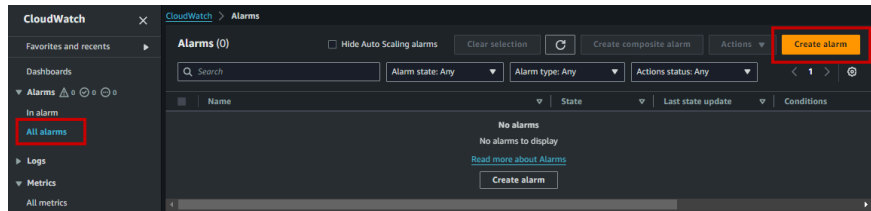
The screenshot shows the AWS RDS console interface for a database instance named 'vpcrdsdbec2connected-1'. The 'Summary' section displays various metrics: DB identifier, Status (Available), Role (Instance), Engine (MySQL Community), Recommendations (2 Informational), CPU (6.39%), Class (db.t3.micro), Current activity (0), and Connections (0). The 'Tags' tab is selected, showing a list of tags with an 'Add' button highlighted by a red box. The 'Add' button is located at the bottom right of the tags section.

- Tag 1 - Scale Up
 - For the key enter `rds_scaling_high_instanceclass`
 - For the value enter the instance class you want the database to scale up to when it meets conditions, i.e. `db.m6gd.xlarge`
 - Click the Add button
- Tag 2 - Scale Down
 - For the key enter `rds_scaling_low_instanceclass`
 - For the value enter the instance class you want the database to scale down to when it meets conditions, i.e. `db.m6gd.large`
 - Click the Add button

Create CloudWatch Alarms

As detailed in the Initial Considerations section of this document it needs to be determined what metric you want to use to scale against. Now that you have the metric selected you need to create a CloudWatch alarm to trigger scaling events.

- Navigate to the [CloudWatch Console](#) in AWS
- Click on the All Alarms item in the left-hand navigation
- Click the Create Alarm button



- Click the Select Metric button
- Click the RDS tile or search for RDS
- Click the DBInstanceIdentifier tile or search for DBInstanceIdentifier
- Search for the metric you want to use for scaling events, and check the checkbox next to it
 - Please note you will see the same metric appear multiple times as you must choose it for the Aurora write instance you want to autoscale
 - For a detailed description of available metrics, please see: <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.AuroraMonitoring.Metrics.html>
- Click the Select Metric button
- Select the desired statistic (Average, Minimum, Maximum etc)
- Select the desired period (10 sec, 30 sec, 1min etc)
- Select your desired threshold type
 - Static - Alarm triggers when the selected metric meets an exact value (i.e. 10)
 - Anomaly Detection - Alarm triggers when the selected metric meets a value in a range (i.e. 1 - 20)
- Select the Alarm Condition (Greater Than, Less Than, Equal etc)
- Enter the Threshold Value
- Click Next
- Click the Add Lambda Action button
- Select the In Alarm radio button for the Alarm State Trigger
- Select the Select Lambda Function radio button
- Select the AimablyAuroraVerticalAutoscaling Lambda function
- Click Next
- Name the alarm and include if it is a scale up alarm or scale down alarm
- Click the Create Alarm button

Add CloudWatch Alarm Tags

You need to create a tag on each scaling alarm you create so that the Lambda function may read them and use them in concert with the database tags to either scale the database up or scale the database down. So for each metric you create you will add a corresponding tag to represent the appropriate action.

- Navigate to the [CloudWatch Console](#) in AWS
- Click on the All Alarms item in the left-hand navigation
- Select the alarm created in the previous section's steps by clicking on the name
- Click on the Tags tab
- Click Manage Tags
- Click Add New Tag
- If the alarm you are working on is a scale-up alarm then you need to add a scale-up tag
 - Enter `rds_scaling_action` as the key (always lowercase)
 - Enter `up` as the value (always lowercase)
- If the alarm you are working on is a scale-down alarm then you need to add a scale-down tag
 - Enter `rds_scaling_action` as the key (always lowercase)
 - Enter `down` as the value (always lowercase)

Database Scaling Testing

Aimably Aurora Write Instance Vertical Autoscaling is designed to allow you to test scaling events by using custom events to trigger scale-up or scale-down actions. To test scaling events:

- Navigate to the [Lambda Console](#) in AWS
- Click on the Functions item in the left-hand navigation
- Locate the AimablyAuroraVerticalAutoscaling function and click on the name
- Click on the Test tab above the Code Source editor
- Select Create New Event for the Test Event Action
- Enter an Event Name, i.e., ScaleUpTest or ScaleDownTest
- Select the Private radio button under Event Sharing Settings
- Select all the existing JSON in the Event JSON editor and delete it
- Copy and paste the below JSON into the Event JSON editor

```
{
  "source": "custom",
  "action": "scaleup",
  "region": "us-east-1",
  "instances": [ "ENTER DATABASE INSTANCE NAME" ]
}
```

- You will want to edit the above JSON to define the scaling action you want to test
 - Enter `scaleup` for a scale-up test or `scaledown` for a scale-down test
 - Enter the region of the RDS instance you want to test
 - Enter the database instance identifier of the RDS instance you want to test
 - Click Test

Code **Test** Monitor Configuration Aliases Versions

Test event [Info](#) Save Test

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Event name

ScaleDownTest

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

cloudwatch-scheduled-event

Event JSON Format JSON

```

1 {
2   "source": "custom",
3   "action": "scaledown",
4   "region": "us-east-2",
5   "instances": [ "arvas-1" ]
6 }
```

- Check for the Executing function: succeeded message

Code **Test** Monitor Configuration Aliases Versions

✓ **Executing function: succeeded** [logs](#)

[Details](#)

Test event [Info](#) Save Test

- Verify that the database instance class is updated in the RDS Console
- You may also test using the AWS Command Line Interface (AWSCLI) using the following command
 - Edit the command text to reflect the correct:
 - Action - either `scaleup` or `scaledown` depending on which you are testing
 - Region - enter the correct region for the Aurora database cluster you are testing, i.e., `us-east1`, `us-west-2`, etc
 - Instances - enter the DB cluster identifier for the database you are testing

```
aws lambda invoke --cli-binary-format raw-in-base64-out --function-name
AimablyAuroraVerticalAutoscaling --invocation-type Event --payload '{"source": "custom",
"action": "ENTER scaleup or scaledown", "region":"ENTER REGION", "instances": [ "ENTER
DATABASE INSTANCE NAME" ] }' response.json
```